



Hochschule
München
University of
Applied Sciences

HOCHSCHULE MÜNCHEN

FAKULTÄT 03
MASCHINENBAU, FAHRZEUGTECHNIK UND FLUGZEUGTECHNIK

Studienarbeit im Rahmen des Moduls Software Engineering und Network Management

Wintersemester 2025/26

Name, Vorname	Studiengang	Matrikel-Nr.	Mikrocontroller
Brösamle, Lucas	MAB	18980619	B-L4S5I-IOT01A

DOZENT: Prof. Dr. rer. nat. Markus KRUG

München, den 15. Januar 2026

Inhaltsverzeichnis

Abkürzungsverzeichnis	2
1 Aufbau und Architektur des Programms	3
1.1 Mikrocontroller-Board und Peripherie	3
1.2 Task-Struktur	3
1.2.1 Task-Koordination	3
1.2.2 Task-Priorisierung	3
1.3 Sensor-Konfiguration	4
1.4 Flussdiagramme der Software-Architektur	4
1.4.1 Gesamtflussdiagramm	4
1.4.2 Einzelsensor-Flow	5
2 Filterung der Sensordaten	7
2.1 Ziel der Filterung	7
2.2 Butterworth-Filter und IIR-Struktur	7
2.3 Parameterwahl	7
2.4 CFT-Partitionierung / Second-Order Sections (SOS)-Filter	8
2.5 STM32-Implementierung	8
2.6 Laufzeitmessung und Performance	8
3 Datenübertragung an den PC / MATLAB	10
3.1 Gesamtablauf der Datenübertragung	10
3.2 Datenformat	10
3.3 Synchronisation des Datenstroms	10
3.4 UART-Übertragung mit DMA	11
3.5 Daten-Transfer-Task	11
3.6 Übertragungszeit der Sensordaten	11
4 MATLAB-Skript zur Visualisierung der Sensordaten	13
4.1 Aufbau und Funktionsumfang der Anwendung	13
4.2 Grafische Benutzeroberfläche	13
4.3 Einlesen der Sensordaten	14
4.4 Datenverarbeitung und Pufferung	14

Abkürzungsverzeichnis

BDU Block Data Update

CPU Central Processing Unit

CTF Canonical Transfer Function

DF2T Direct Form II Transposed

DMA Direct Memory Access

DWT Data Watchpoint and Trace

freeRTOS Free Real-Time Operating System

GUI Graphical User Interface

I2C Inter-Integrated Circuit

IIR Infinite Impulse Response

IMU Inertial Measurement Unit

ODR Output Data Rate

SOS Second-Order Sections

UART Universal Asynchronous Receiver Transmitter

1. Aufbau und Architektur des Programms

Die Software läuft auf dem Mikrocontroller-Board **B-L4S5I-IOT01A**, das Rechenleistung, Speicher und Peripherie für Sensordatenerfassung, Echtzeitverarbeitung und Datenübertragung bereitstellt. Verwendet werden insbesondere Inter-Integrated Circuit (I2C)- und Universal Asynchronous Receiver Transmitter (UART)-Schnittstellen, Direct Memory Access (DMA)-Controller sowie Interrupt-Funktionalitäten.

1.1 Mikrocontroller-Board und Peripherie

- **I2C:** Anbindung von *LSM6DSL* (Beschleunigung/Gyroskop, Inertial Measurement Unit (IMU)), *LIS3MDL* (Magnetometer), *LPS22HB* (Barometer) und *HTS221* (Feuchtigkeit). DMA unterstützt die Kommunikation und entlastet die Central Processing Unit (CPU).
- **UART:** Datenübertragung an externe Systeme inkl. Logging. Ebenfalls DMA-unterstützt.
- **Interrupts:** Sensorinterne Interrupts aktivieren periodische Tasks sofort.

1.2 Task-Struktur

Jeder Sensor wird in einem eigenen Free Real-Time Operating System (freeRTOS)-Task verarbeitet. Hochfrequente Sensoren (IMU, Magnetometer) erhalten höhere Priorität, langsamere Sensoren (Barometer, Feuchtigkeit) niedrigere. Die Filterung der Sensordaten sowie die Datenübertragung wird in eigenen Tasks realisiert. Dadurch wird die Echtzeitverarbeitung nicht durch lange Übertragungen blockiert.

1.2.1 Task-Koordination

- **Semaphore:** Schutz von I2C- und UART-Bussen gegen parallele Zugriffe.
- **Thread-Flags/Event-Groups:** Aktivierung von Tasks bei neuen Sensordaten.
- **Interrupt-Service-Routinen:** Zeitkritische Ereignisse, z.B. Datenbereit-Interrupts, werden direkt verarbeitet.

1.2.2 Task-Priorisierung

Die Priorität der einzelnen Task wird durch die folgenden Kriterien festgelegt:

- Abtastrate der Sensoren
- Rechenaufwand
- Echtzeitkritikalität

Alle verwendeten Tasks mit Namen, zugehörigem Sensor oder Funktion sowie der Priorität sind in nachfolgender Tabelle dargestellt:

Durch diese Struktur werden potenzielle Blockaden zwischen Tasks mit hoher Ausführungsfrequenz vermieden. Niedrig priorisierte Tasks mit geringer Abtastrate haben keinen Einfluss auf die Echtzeitverarbeitung und beanspruchen nur minimale Rechenressourcen.

Tabelle 1.1: Übersicht der implementierten Tasks und deren Prioritäten

Task	Sensor / Funktion	Priorität / freeRTOS
InitTask	Systeminitialisierung	High
MagnetoTask	LIS3MDL (Magnetometer, 155 Hz)	Above Normal 2
IMUTask	LSM6DSL (Beschleunigung/Gyroskop, 104 Hz)	Above Normal 1
ToFTask	VL53L0X (Time-of-Flight-Sensor, 5 Hz)	Normal 2
BaroTask	LPS22HB (Drucksensor, 1 Hz)	Below Normal 1
HumidityTask	HTS221 (Feuchte und Temperatur, 1 Hz)	Below Normal 1
DataTransmitTask	UART-Datenübertragung	Low 1
DataFilterTask	Sensor-Datenfilterung	Low 2

1.3 Sensor-Konfiguration

Die Sensoren werden jeweils mit den spezifizierten Messbereichen und Abtastraten konfiguriert. Die Umrechnung der erfassten Rohwerte in die entsprechenden SI-Einheiten erfolgt direkt im jeweiligen Sensor-Treiber.

LSM6DSL: Der Inertialsensor wird mit einem Beschleunigungsbereich von $\pm 2g$, einem Gyroskopbereich von $\pm 250^\circ/s$ und einer Output Data Rate (ODR) von 104 Hz betrieben.

LIS3MDL: Das Magnetometer ist auf einen Messbereich von ± 16 Gauss bei einer ODR von 155 Hz konfiguriert, wobei auch der integrierte Temperatursensor aktiviert ist.

LPS22HB: Der barometrische Drucksensor arbeitet mit einer ODR von 1 Hz.

HTS221: Der Feuchte- und Temperatursensor wird mit einer ODR von 1 Hz betrieben, wobei die Block Data Update (BDU)-Funktion aktiviert ist.

VL53L0X: Der Distanzsensord wird im Continuous-Ranging-Modus mit einer ODR von 5 Hz betrieben.

Die Rohwerte der einzelnen Sensoren werden jeweils in ihrem eigenen Treiber in SI-Einheiten umgerechnet. Dies ermöglicht eine einheitliche Weiterverarbeitung der Messdaten in physikalisch interpretierbaren Größen und reduziert den Implementierungsaufwand in den nachfolgenden Tasks.

1.4 Flussdiagramme der Software-Architektur

1.4.1 Gesamtflussdiagramm

Das vereinfachte Gesamtflussdiagramm stellt den grundlegenden Ablauf der implementierten Software-Architektur dar, wie in Abbildung 1.1 dargestellt. Es zeigt die Abfolge von der Initialisierung des Mikrocontrollers und der angebundenen Peripherie über die zyklische Erfassung der Sensordaten bis hin zur Filterung und anschließenden Datenübertragung. Ziel der Darstellung ist es, die zentralen Verarbeitungsschritte sowie deren zeitliche und logische Abhängigkeiten übersichtlich darzustellen.

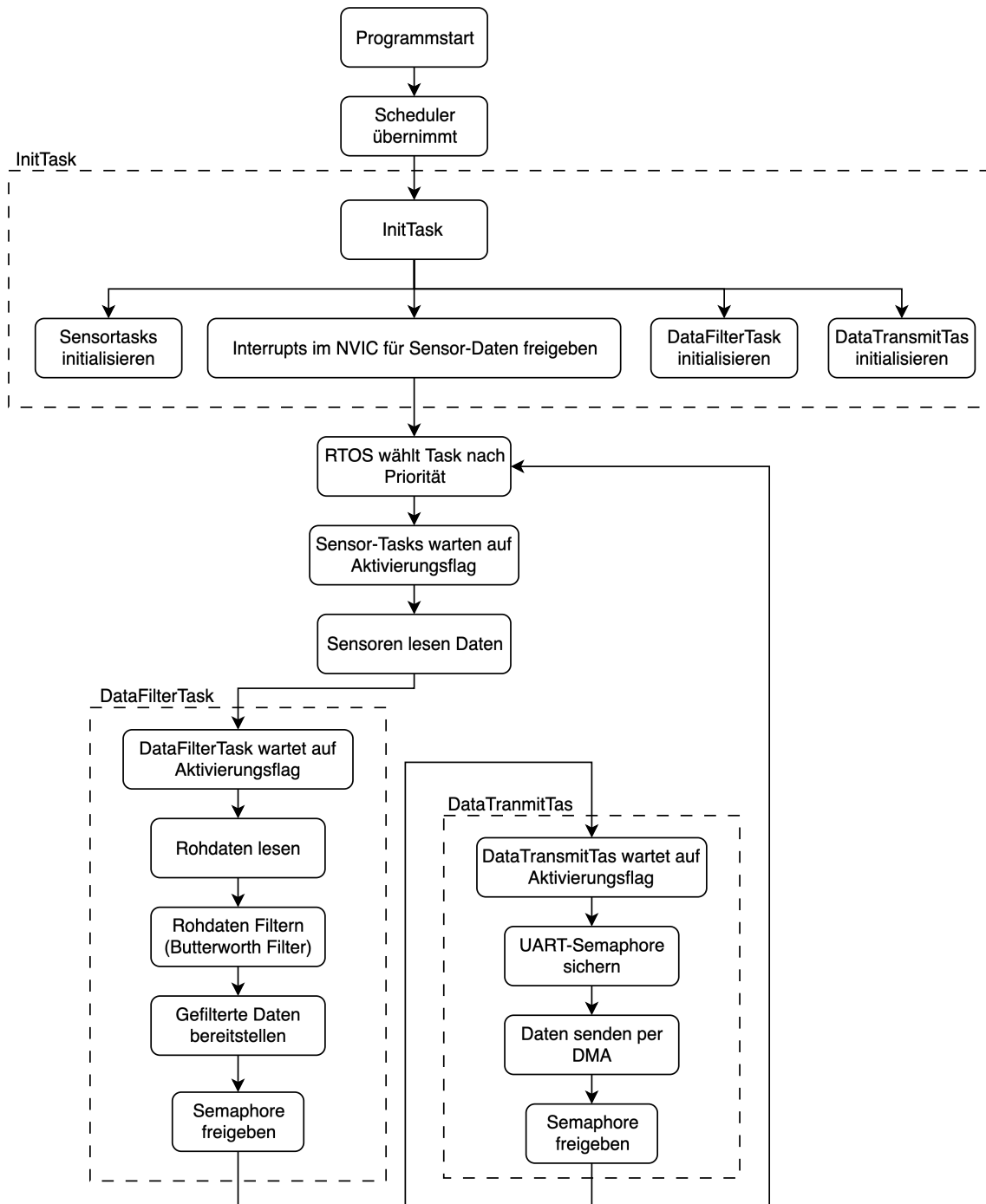


Abbildung 1.1: Vereinfachtes Flussdiagramm der Software-Architektur

1.4.2 Einzelsensor-Flow

Das detaillierte Flussdiagramm eines einzelnen Sensors verdeutlicht den internen Ablauf der Sensordatenverarbeitung, wie in Abbildung 1.2 dargestellt. Beginnend bei der Initialisierung und Konfiguration des Sensors werden die einzelnen Schritte der Datenerfassung sowie der Umrechnung der Rohwerte in SI-Einheiten dargestellt.

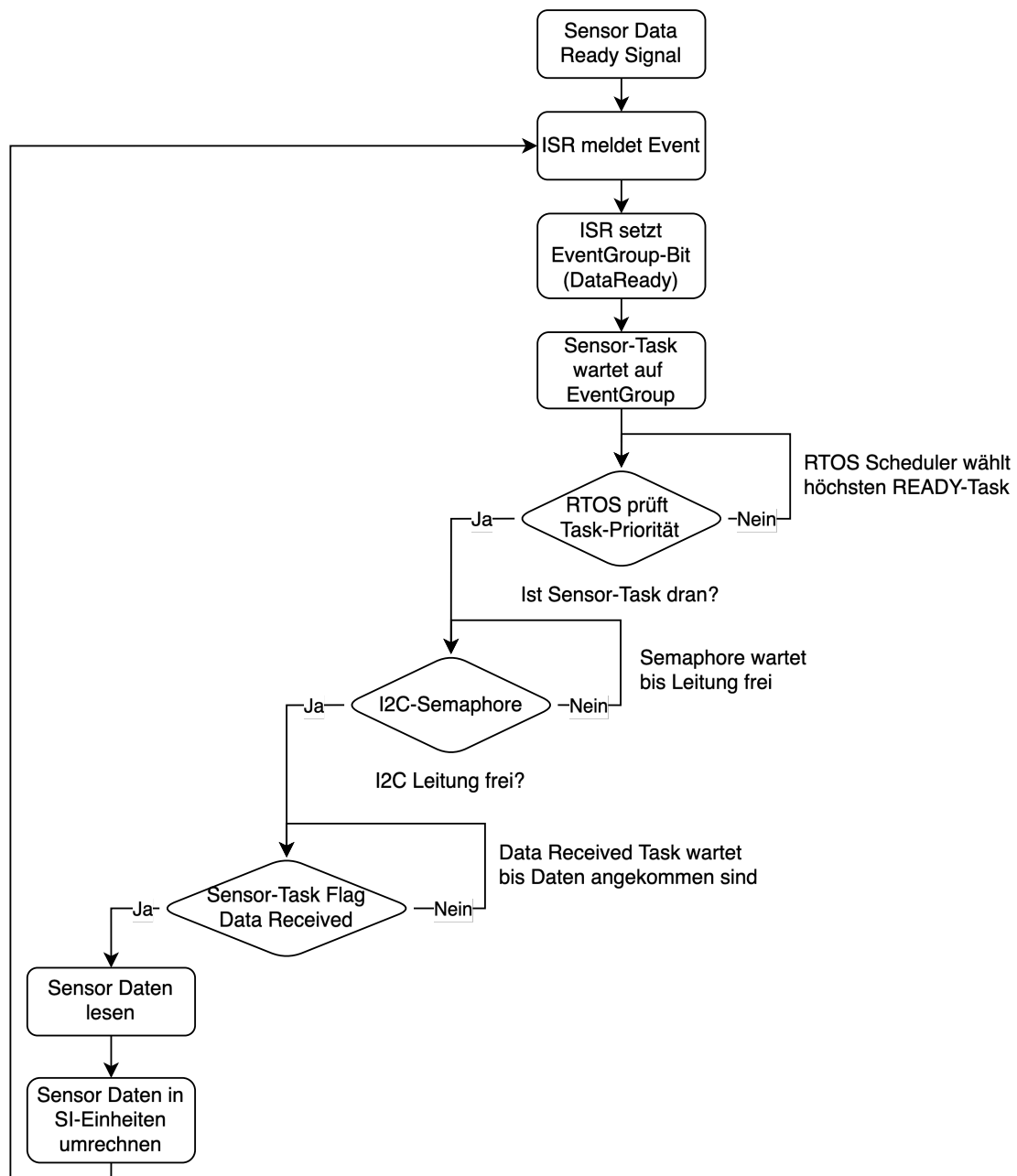


Abbildung 1.2: Flussdiagramm der Verarbeitung eines einzelnen Sensors

2. Filterung der Sensordaten

2.1 Ziel der Filterung

Zur Reduzierung hochfrequenten Rauschens auf den Sensordaten des *LSM6DSL* (Beschleunigung / Gyroskop) und *LIS3MDL* (Magnetometer) wird auf dem Mikrocontroller-Board ein digitaler Butterworth-Filter fünfter Ordnung implementiert. Alle anderen Sensorwerte, wie Barometer- oder Feuchtigkeitssensor, werden lediglich in globalen Variablen auf dem Board gespeichert.

2.2 Butterworth-Filter und IIR-Struktur

Butterworth-Filter zählen zu den Infinite Impulse Response (IIR)-Filtern und besitzen eine glatte Amplitudencharakteristik im Durchlassbereich. Zur stabilen Umsetzung wird der Filter in mehrere SOS zerlegt und als Canonical Transfer Function (CTF)-partitionierter Filter realisiert.

2.3 Parameterwahl

Für die Berechnung der Filterkoeffizienten des 5. Ordnung Butterworth-Filters wurden folgende Parameter gewählt:

- **Samplingfrequenz:** $f_s = 100$ Hz (*LSM6DSL*), $f_s = 155$ Hz (*LIS3MDL*)
- **Cutoff-Frequenz:** $f_c = 10$ Hz, gewählt entsprechend der maximalen Signalbandbreite der Sensoren
- **Normierte Grenzfrequenz:** $W_n = f_c/(f_s/2)$

Die Samplingfrequenzen f_s entsprechen den nativen Abtastraten der Sensoren, wodurch keine zusätzlichen Verzögerungen oder Resampling erforderlich sind. Die Cutoff-Frequenz $f_c = 10$ Hz wurde so gewählt, dass hochfrequentes Rauschen effektiv unterdrückt wird, ohne die relevanten Signalanteile der Sensoren zu dämpfen. Die normierte Grenzfrequenz W_n stellt sicher, dass der digitale Filter korrekt auf die jeweilige Samplingfrequenz skaliert ist.

Die Filterkoeffizienten wurden in MATLAB mit der Funktion `butter` berechnet und anschließend in SOS umgewandelt, um eine numerisch stabile Umsetzung auf dem Mikrocontroller zu gewährleisten. Jede Biquad-Sektion enthält dabei die Koeffizienten $[b_0, b_1, b_2, a_0, a_1, a_2]$, die in der CTF-partitionierten Filterstruktur verwendet werden. Im Folgenden wird der MATLAB-Code zur Berechnung der Filterparameter dargestellt:

```
% Parameter
fs = 100; fc = 10;           % Sampling freq. und Cutoff freq.
Wn = fc / (fs/2);           % Normierte Grenzfrequenz

% Butterworth-Filter 5. Ordnung
[b,a] = butter(5, Wn);       % Direct Form coefficients

% Umwandlung in Second-Order Sections (SOS)
sos = tf2sos(b,a);           % Jede Zeile: [b0 b1 b2 a0 a1 a2]
g = 1;                       % Gesamtverstärkung
```

Dieses Vorgehen stellt sicher, dass der Filter auf dem Mikrocontroller effizient und stabil implementiert werden kann, während hochfrequentes Rauschen zuverlässig unterdrückt wird.

2.4 CFT-Partitionierung / SOS-Filter

Die Filterung der Sensordaten erfolgt in SOS-Struktur. Für einen Filter 5. Ordnung werden pro Sensorachse drei Biquad-Sektionen hintereinandergeschaltet. Jede Sektion wird nach der Direct Form II Transposed (DF2T)-Struktur implementiert, um Speicherbedarf zu minimieren und die numerische Stabilität bei der digitalen Filterung zu verbessern. Die Berechnung einer einzelnen Biquad-Sektion erfolgt gemäß der DF2T-Formel:

$$y[n] = b_0w[n] + b_1w[n-1] + b_2w[n-2], \quad w[n] = x[n] - a_1w[n-1] - a_2w[n-2] \quad (2.1)$$

2.5 STM32-Implementierung

Auf dem Mikrocontroller werden separate Biquad-Ketten für jede Achse der freeRTOS-Sensoren (*LSM6DSL*) und des Magnetometers (*LIS3MDL*) initialisiert und verarbeitet. Die eigentliche Filterung der Sensordaten erfolgt zentral im Task `taskFilterData`. Dieser ruft die Funktionen aus den separaten Implementierungen `filter.c` und `filter.h` auf, welche die Rohwerte sequentiell durch die hintereinandergeschalteten Biquad-Sektionen leiten und die gefilterten Ergebnisse in globalen Variablen ablegen.

Die Rohwerte selbst werden von den Sensor-Treibern bereitgestellt, die die Sensoren konfigurieren und die Rohwerte in SI-Einheiten umrechnen. `filter.c` stellt ausschließlich die Filterlogik bereit.

- **Init:** Initialisierung aller Biquad-Ketten mit den SOS-Koeffizienten.
- **Update:** Verarbeitung neuer Rohdaten durch die drei Biquad-Sektionen jeder Achse.
- **Ergebnis:** Gefilterte Werte werden in den globalen Strukturen `lsm6dsl_filtered_values` bzw. `lis3mdl_filtered_values` abgelegt.

Diese Implementierung ermöglicht eine entkoppelte und deterministische Filterung, ohne die Ausführung anderer Tasks auf dem Mikrocontroller zu blockieren.

2.6 Laufzeitmessung und Performance

Um die maximale Laufzeit der Filterfunktion zu ermitteln, wurde der Data Watchpoint and Trace (DWT) Cycle Counter des Cortex-M-Mikrocontrollers verwendet. Diese Methode erlaubt eine sehr genaue Messung der Anzahl von Mikrocontroller-Takten, die für die Berechnung der gefilterten Werte benötigt werden. Der DWT-Counter wird direkt vor und nach dem Aufruf der Update-Funktion der Filterbibliothek gestartet und gestoppt, wie im folgenden Ausschnitt aus `Data_Filter_Task` dargestellt:

```
__disable_irq();
start = DWT->CYCCNT;

LSM6DSL_Filter_Update(lsm6dsl_values);

stop = DWT->CYCCNT;
__enable_irq();

uint32_t cycles = stop - start;
runtime_us_acc = (float)cycles / SystemCoreClock * 1e6;
```

Die gemessenen maximalen Laufzeiten für die Sensorfilter sind wie folgt:

- **LSM6DSL-Filter (freeRTOS, Achse X):** `max_cycles_acc` = 2914, Laufzeit: 24.26 μ s
- **LIS3MDL-Filter (Magnetometer, Achse X):** `max_cycles_mag` = 1551, Laufzeit: 12.92 μ s

Interpretation: Die gemessenen Laufzeiten liegen deutlich unter den jeweiligen Abtastperioden der Sensoren (*LSM6DSL*: 9.6 ms, *LIS3MDL*: 6.45 ms). Damit kann die Filterung deterministisch in Echtzeit durchgeführt werden, ohne dass Datenpunkte verloren gehen. Der Unterschied in den Laufzeiten erklärt sich aus der Anzahl der zu verarbeitenden Daten: Der *LSM6DSL*-Filter bearbeitet 6 Kanäle (3 Achsen Beschleunigung + 3 Achsen Gyroskop), während der *LIS3MDL*-Filter nur 3 Magnetometer-Werte verarbeitet. Daher ist die Laufzeit des *LIS3MDL*-Filters etwa halb so groß wie die des *freeRTOS*-Filters.

3. Datenübertragung an den PC / MATLAB

Die vom Mikrocontroller erfassten Sensordaten werden zyklisch an einen PC übertragen, um dort in MATLAB visualisiert zu werden. Die Implementierung basiert auf einer ereignisgesteuerten Task-Architektur unter freeRTOS und ist auf eine effiziente, deterministische und robuste Datenübertragung ausgelegt. Zur Minimierung der CPU-Last wird die serielle Kommunikation über UART vollständig mittels DMA realisiert.

3.1 Gesamtablauf der Datenübertragung

Die Datenübertragung erfolgt nicht periodisch, sondern ereignisgesteuert. Sobald neue Sensordaten vorliegen, wird eine Verarbeitungskette ausgelöst, die aus folgenden Schritten besteht:

- Zunächst signalisieren die Sensor-Tasks das Vorliegen neuer Rohdaten.
- Diese werden anschließend zentral in einer Filter-Task verarbeitet.
- Nach Abschluss der Filterung werden die Roh- und Filterdaten zusammengefasst und in einem definierten Datenformat über UART an den PC übertragen.

Durch diese Entkopplung der einzelnen Verarbeitungsschritte wird sichergestellt, dass zeitkritische Sensorabfragen und rechenintensive Filteroperationen die Datenübertragung nicht blockieren.

3.2 Datenformat

Die Übertragung erfolgt in Form eines binären Datenframes, dessen Aufbau in einem strukturierten Datentyp definiert ist. Alle Sensordaten werden als 32-Bit IEEE 754 Fließkommazahlen `float` übertragen. Zur eindeutigen Synchronisation des seriellen Datenstroms enthält jedes Datenpaket ein 32-Bit-Delimiter-Feld vom Typ `uint32_t` mit dem festen Wert `0xDEADBEEF`. Dieses Kennwort markiert den Beginn eines neuen Datenframes.

Der vollständige Aufbau des übertragenen Datenpakets ist in Tabelle 3.1 dargestellt.

Tabelle 3.1: Aufbau des Datenpakets für die UART-Übertragung

Feldname	Datentyp	Größe [Byte]
<code>delimiter</code>	<code>uint32_t</code>	4
<code>acc_x</code> , <code>acc_y</code> , <code>acc_z</code>	<code>float</code>	12
<code>gyro_x</code> , <code>gyro_y</code> , <code>gyro_z</code>	<code>float</code>	12
<code>mag_x</code> , <code>mag_y</code> , <code>mag_z</code>	<code>float</code>	12
<code>acc_x.filtered</code> , <code>acc_y.filtered</code> , <code>acc_z.filtered</code>	<code>float</code>	12
<code>gyro_x.filtered</code> , <code>gyro_y.filtered</code> , <code>gyro_z.filtered</code>	<code>float</code>	12
<code>mag_x.filtered</code> , <code>mag_y.filtered</code> , <code>mag_z.filtered</code>	<code>float</code>	12

3.3 Synchronisation des Datenstroms

Da UART eine kontinuierliche Bytefolge ohne feste Paketgrenzen überträgt, ist eine explizite Rahmensynchronisation erforderlich. Der Empfänger überprüft beim Einlesen der Daten fortlaufend das Delimiter-Feld. Wird das erwartete Bitmuster erkannt, werden die nachfolgenden Bytes als vollständiges Datenpaket interpretiert. Fehlerhafte oder unvollständige Frames werden verworfen, sodass eine stabile Synchronisation auch bei verspätetem Start des Empfängers oder bei Übertragungsfehlern

gewährleistet ist.

Der Delimiter ist bewusst als Ganzzahl und nicht als Fließkommazahl definiert, da nur so ein eindeutiges und unveränderliches Bitmuster garantiert werden kann. Fließkommazahlen sind für diesen Zweck ungeeignet, da sie keine verlässliche binäre Repräsentation besitzen.

3.4 UART-Übertragung mit DMA

Die serielle Datenübertragung erfolgt mit einer Baudrate von 115200 Bit/s über UART. Zur Entlastung der CPU wird die Übertragung vollständig mittels DMA realisiert. Dabei werden die Daten direkt aus dem Speicher an das UART-Peripheriegerät übertragen, ohne dass die CPU byteweise eingreifen muss. Die UART-Ressource wird durch ein Semaphore geschützt, um konkurrierende Zugriffe anderer Tasks zu verhindern.

3.5 Daten-Transfer-Task

Der Daten-Transfer-Task `DataTransmitTask` übernimmt die Übertragung der Sensordaten über UART. Der Task wird nicht zyklisch ausgeführt, sondern blockiert in einer Endlosschleife auf das Thread-Flag `FILTERED_DATA_READY_FLAG`, das nach der Fertigstellung der Filterung durch `DataFilterTask` gesetzt wird.

Wie im abgebildeten Codeausschnitt zu sehen, werden nach der Aktivierung zunächst alle Roh- und gefilterten Sensordaten aus den globalen Strukturen in das Übertragungspaket `transmit_data` kopiert. Für die IMU (Beschleunigung + Gyroskop) werden sechs Werte mit `memcpy` übertragen, für das Magnetometer drei Werte:

```
memcpy(&transmit_data.acc_x, &lsm6dsl_values.acc_x, 6 * sizeof(float)); // acc
+ gyro
memcpy(&transmit_data.mag_x, &lis3mdl_values.x, 3 * sizeof(float));

memcpy(&transmit_data.acc_x_filtered, &lsm6dsl_filtered_values.acc_x, 6 * sizeof
(float));
memcpy(&transmit_data.mag_x_filtered, &lis3mdl_filtered_values.x, 3 * sizeof(
float));
```

Vor der Übertragung wird die UART-Semaphore `UART1availableHandle` gesichert, um parallele Zugriffe auf den Bus zu verhindern:

```
osSemaphoreAcquire(UART1availableHandle, osWaitForever);
```

Anschließend wird die Datenübertragung über DMA gestartet:

```
HAL_UART_Transmit_DMA(&huart1, (const uint8_t *)&transmit_data, sizeof(
transmit_data));
```

Während der blockweisen Übertragung wird der Halb-Transfer-Interrupt deaktiviert, da keine Zwischenauswertung der Daten erforderlich ist:

```
__HAL_DMA_DISABLE_IT(huart1.hdmatx, DMA_IT_HT);
```

Diese Vorgehensweise stellt sicher, dass die Sensordaten zuverlässig und deterministisch übertragen werden, während die Echtzeitverarbeitung in den Filter-Tasks weiterhin ungehindert erfolgen kann.

3.6 Übertragungszeit der Sensordaten

Die Übertragungszeit eines Datenframes vom Mikrocontroller zum PC über UART lässt sich direkt aus der Framegröße und der konfigurierten Baudrate berechnen. Jeder Datenframe besteht aus insgesamt 76 Byte, bestehend aus Roh- und gefilterten Sensordaten für Beschleunigung, Gyroskop und

Magnetometer sowie einem 32-Bit-Delimiter.

Bei einer UART-Konfiguration von 115200 Bd mit 8 Datenbits, 1 Startbit und 1 Stoppbit (8N1) werden für jedes Byte 10 Bit übertragen. Die Gesamtanzahl der zu übertragenden Bits pro Frame beträgt somit:

$$N_{\text{Bit}} = 76 \cdot 10 = 760 \text{ Bit.}$$

Die theoretische Übertragungszeit t_{UART} ergibt sich aus

$$t_{\text{UART}} = \frac{N_{\text{Bit}}}{\text{Baudrate}} = \frac{760}{115200} \approx 6,6 \text{ ms.}$$

Da die Übertragung über DMA erfolgt, blockiert sie die CPU nicht, sondern stellt lediglich die Belegungsdauer der UART-Schnittstelle dar. Diese kurze Übertragungszeit stellt sicher, dass die Sensordaten nahezu in Echtzeit an den PC übertragen werden können, ohne die Filter- und Verarbeitungstasks auf dem Mikrocontroller zu verzögern.

4. MATLAB-Skript zur Visualisierung der Sensordaten

Zur Visualisierung und Analyse der vom Mikrocontroller übertragenen Sensor-Daten wird eine MATLAB-Anwendung verwendet. Ziel der Anwendung ist die übersichtliche Echtzeitdarstellung der Roh- und gefilterten Sensordaten sowie die Anzeige von Statusinformationen zum Datenempfang.

4.1 Aufbau und Funktionsumfang der Anwendung

Die Anwendung ist objektorientiert als MATLAB-Klasse implementiert und kombiniert eine grafische Graphical User Interface (GUI) mit einer kontinuierlichen seriellen Datenerfassung. Die wesentlichen Funktionen sind:

- Echtzeitdarstellung der Sensordaten in jeweils einem Diagramm pro Sensor
- Gleichzeitige Anzeige von Rohwerten und gefilterten Werten
- Status- und Logausgaben zum Verbindungszustand und Datenempfang
- Anzeige der seit Programmstart empfangenen Datenmenge in Bytes

4.2 Grafische Benutzeroberfläche

Die grafische Benutzeroberfläche wird mit klassischen MATLAB-GUI-Komponenten wie `figure`, `subplot` und `uicontrol` realisiert. Abbildung 4.1 zeigt den Aufbau der Oberfläche.



Abbildung 4.1: Aufbau der MATLAB-GUI für die Messdatenvisualisierung

- **Diagrammbereich:** Drei Diagramme zur Darstellung der Beschleunigung, der Winkelgeschwindigkeit und des Magnetfeldes. In jedem Diagramm werden die Rohwerte sowie die gefilterten Werte der jeweiligen Sensorachsen dargestellt. Die Rohwerte werden als Volllinie dargestellt und die gefilterten Werte gestrichelt.
- **Steuerung:** Über die Schaltflächen **Verbinden** und **Stop** kann die serielle Verbindung gestartet bzw. beendet werden.
- **Status- und Logbereich:** Ein Statusfeld informiert über den aktuellen Zustand der Verbindung. Zusätzlich werden relevante Ereignisse, wie Verbindungsaufbau oder Fehler, mit Zeitstempel in einem Logfenster ausgegeben.
- **Byte-Zähler:** Ein separates Textfeld zeigt die insgesamt empfangene Datenmenge in Bytes seit dem letzten Programmstart an.

4.3 Einlesen der Sensordaten

Die Kommunikation mit dem Mikrocontroller erfolgt über eine serielle Schnittstelle mithilfe der MATLAB-Funktion `serialport`. Nach dem Aufbau der Verbindung werden kontinuierlich Datenframes empfangen. Jedes Datenpaket besitzt einen festen Aufbau:

- 4 Byte Delimiter zur Synchronisation
- 18 Gleitkommazahlen (`single`) für Sensor- und Filterdaten

Zur Sicherstellung der Synchronität wird der Delimiter jedes empfangenen Frames überprüft. Ungültige Daten werden verworfen, bis wieder ein korrektes Paket erkannt wird.

4.4 Datenverarbeitung und Pufferung

Nach erfolgreichem Einlesen werden die Daten in Rohwerte und gefilterte Werte für Beschleunigung, Gyroskop und Magnetometer aufgeteilt. Die Messwerte werden in Ringpuffern gespeichert, um eine zeitlich begrenzte Historie für die Darstellung zu erhalten.

Für das Magnetometer erfolgt zusätzlich eine Anpassung des Koordinatensystems durch Invertierung der X- und Y-Achse. Dadurch wird eine konsistente Ausrichtung mit den übrigen Sensoren erreicht. Die Diagramme werden nicht bei jedem empfangenen Datenpaket aktualisiert, sondern in festgelegten Intervallen. Dies reduziert die Rechenlast und sorgt für eine flüssige Darstellung der Messdaten in der GUI. Die Rohwerte werden als Volllinie dargestellt, die gefilterten Werte gestrichelt.